

AND/OR 木の探索

準備

- [...]はリストを表す.
 - パス(経路)は, ノードのリストで表す.
 - ノード x を根とする(部分)木は, $[x, (x \text{ の子ノードを根とする部分木のリスト})]$ で表す.
(各分岐が AND 分岐か OR 分岐かは, 別途与えられるものとする.)
- 変数 L はリスト, 変数 X, V, W は単一のノードを保持する.
- $+$ はリスト連結の演算子とする.
- $next(x)$ は, x の子ノードのリストを返す関数とする.

AND/OR 木探索の再帰関数

$aos(X)$: // X を根とする AND/OR 木を探索し X が成功(1)か失敗(0)かを返す

```

1  if (X が末端で成功) return 1;
2  if (X が末端で失敗) return 0;
3  if (X が OR 分岐)
    3.1  for each V in next(X) do       // すべての(子)部分木について,
        3.1.1 if (aos(V) ≠ 0) return 1; // 一つでも成功があれば成功とする.
    3.2  return 0;                       // すべて失敗なら失敗とする.
4  if (X が AND 分岐)
    4.1  for each V in next(X) do       // すべての(子)部分木について,
        4.1.1 if (aos(V) = 0) return 0; // 一つでも失敗があれば失敗とする.
    4.2  return 1;                       // すべて成功なら成功とする.
```

※ 成功部分木を返すようにするには, 例えば以下のようにするとよい.

$aos2(X)$: // X を根とする AND/OR 木を探索し成功部分木か 0(失敗)を返す

```

1  if (X が末端で成功) return [X];
2  if (X が末端で失敗) return 0;
3  if (X が OR 分岐)
    3.1  for each V in next(X) do
        3.1.1  $W \leftarrow aos2(V)$ ;
        3.1.2 if ( $W \neq 0$ ) return [X] + [W];
    3.2  return 0;
4  if (X が AND 分岐)
    4.1   $L \leftarrow []$ ;
    4.2  for each V in next(X) do
        4.2.1  $W \leftarrow aos2(V)$ ;
        4.2.2 if ( $W = 0$ ) return 0;
        4.2.3 else  $L \leftarrow L + [W]$ ;
    4.3  return [X] + L;
```

〔参考〕木の縦型探索の再帰関数

tdfs(X): // X を根とする木を探索し、見つけた目標ノードを返す

```

1  if (X が目標状態) return X;
2  for each V in next(X) do     // すべての子ノードについて,
    2.1  W ← tdfs(V);         // その子ノードを根とする部分木を探索し,
    2.2  if (W ≠ 0) return W;   // 目標ノードが見つかればそれを返す.
3  return 0;                   // 見つからなければ失敗とする.
```

※ 根からのパス(経路)を返すようにするには, 例えば以下のようにするとよい.

tdfs2(X): // X を根とする木を探索し、見つけた目標ノードまでのパスを返す

```

1  if (X が目標状態) return [X];
2  for each V in next(X) do
    2.1  W ← tdfs2(V);
    2.2  if (W ≠ 0) return [X]+W;
3  return 0;
```